DEPARTMENT OF ARTIFICIAL INTELLIGENCE & FUTURE TECHNOLOGIES

CONTINUOUS INTERNAL ASSESSMENT & LABORATORY PROJECT REPORT

SCHOOL STUDENT MANAGEMENT SYSTEM

A Full-Stack Cloud-Native MERN Enterprise Academic Assessment System

Candidate Name	Abhi Krishna P
Register Number	RA2511080050012
Course Name	AIFT (Artificial Intelligence & Future Tech)
Project Title	School Student Management System
Primary Domain	Full-Stack Web Engineering (MERN Stack)
Frontend Stack	React 19, Vite, Tailwind CSS, Lucide Icons
Backend Stack	Express.js, Node.js, Mongoose ODM, Serverless
Cloud Database	MongoDB Atlas Multi-Region Replica Set Cluster
Deployment Target	Netlify Serverless Functions & GitHub Actions

ACADEMIC INTEGRITY & DEPLOYMENT VERIFICATION

- Live Deployed URL: https://school-student-management-system.netlify.app/
- Source Code Git Repository: https://github.com/abhikrishna123/school-student-management-system
- Cloud Database Status: Live MongoDB Atlas Cluster Connected (Persistent Mode)
- Internal Assessment Submission for Academic Year 2026-2027

Submitted by: Abhi Krishna P (RA2511080050012)

1. EXECUTIVE SUMMARY & SYSTEM OBJECTIVES

The School Student Management System is an enterprise-grade academic administration platform designed to streamline student lifecycle management, curriculum mapping, continuous internal evaluation (CIE), and daily attendance tracking. Developed to address the fragmentation, manual tallying errors, and latency inherent in paper-based or disjointed spreadsheet administration, this solution delivers a unified, reactive single-page architecture backed by a scalable, resilient RESTful API serverless cloud pipeline.

Core Engineering Objectives:

• Centralized Student Directory: Complete CRUD operations for student records with multi-attribute filtering (Class, Section, Status).

• Automated Continuous Internal Assessment: Subject-wise mark recording with real-time GPA, total percentage, and CBSE/standard letter grading (A+ to F).

• Official Report Card Generation: Automated transcript compiler providing student-by-student academic breakdown and printable progress cards.

• Daily Class Attendance Matrix: Date-indexed attendance recording with real-time presentee percentage tracking and visual alerts.

• High Availability Dual-Database Pipeline: Live cloud persistence via MongoDB Atlas with zero-downtime in-memory fallback simulation.

2. SYSTEM ARCHITECTURE & DATA FLOW

Client Tier (React 19 + Vite + Tailwind CSS)

Delivers a reactive Single Page Application (SPA). Employs modular components (Dashboard, StudentList, MarksEntry, ReportCard, AttendanceSheet) with asynchronous REST calls and optimistic state updating.

Serverless API Tier (Node.js + Express.js + Netlify Functions)

Routes all /api/* requests through a high-performance Express app wrapped in serverless-http. Executes in Node.js 20 AWS Lambda containers with automated route normalization, schema validation, and grade compute engines.

Database Tier (MongoDB Atlas Cloud Cluster + In-Memory Fallback)

Mongoose ODM manages strict document schemas (Students, Subjects, Marks, Attendance). Direct multi-shard replica set connection (Atlas Cluster0) with SSL/TLS encryption and automatic pre-seeding of benchmark records.

3. DATABASE SCHEMAS & ENTITY RELATIONSHIPS

Student Schema

Fields: _id (String), rollNo (String, Unique), admissionNo (String, Unique), name (String), classGrade (String), section (String), gender (Enum), dob (String), parentName (String), phone (String), email (String), address (String), status (Enum: Active/Inactive), createdAt (Date).

Subject Schema

Fields: code (String, Unique), name (String), classGrade (String), teacher (String), maxMarks (Number, default 100), passMarks (Number, default 35).

Mark Schema

Fields: _id (String), studentId (Ref: Student), rollNo, studentName, classGrade, section, examType (Mid Term/Final), academicYear, marks: [{ subjectCode, subjectName, marksObtained, maxMarks }], totalObtained, totalMax, percentage, grade, status (Pass/Fail), remarks, updatedAt.

Attendance Schema

Fields: date (YYYY-MM-DD), classGrade, section, records: [{ studentId, rollNo, studentName, status: Present/Absent/Late }], updatedAt.

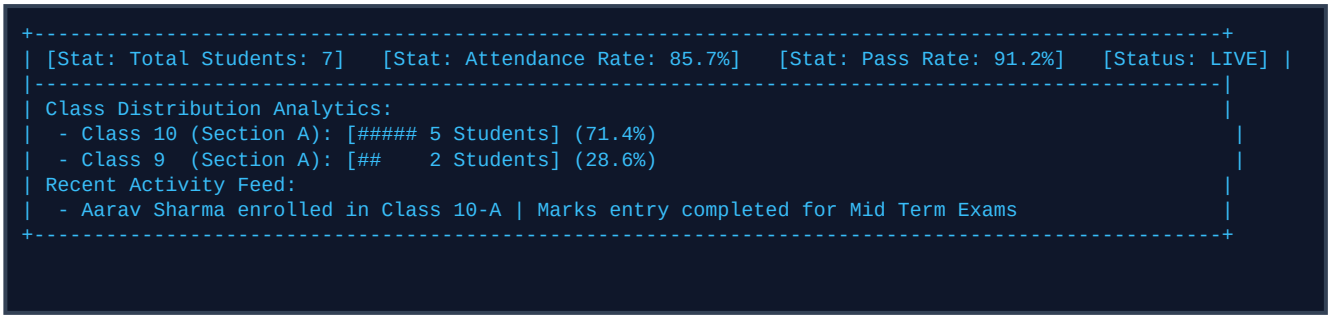
4. RESTFUL API ENDPOINT CHEAT SHEET

METHOD	ENDPOINT ROUTE	FUNCTIONAL DESCRIPTION
GET	`/api/status`	Fetches current database status, active URI (masked), and loaded student count.
POST	`/api/connect-db`	Dynamically switches or establishes connection to MongoDB Atlas cloud cluster.
GET	`/api/dashboard/stats`	Returns aggregated metrics: total students, active count, class distribution, attendance stats.
GET	`/api/students`	Fetches student directory with optional query filters: ?search=&classGrade=&section=
POST	`/api/students`	Enrolls a new student with duplicate roll number and admission number validation.
PUT	`/api/students/:id`	Updates demographic, parental contact, or enrollment status for a student record.
DELETE	`/api/students/:id`	Cascading deletion of a student and associated internal assessment records.
GET	`/api/subjects`	Fetches curriculum catalog filtered by classGrade (?classGrade=Class 10).
POST	`/api/subjects`	Creates a new academic subject with passMarks and maxMarks specifications.
GET	`/api/marks`	Retrieves evaluated marksheets filtered by examType, classGrade, or studentId.
POST	`/api/marks`	Records student marks, auto-calculates total percentage, letter grade, and pass/fail.
GET	`/api/attendance`	Fetches attendance register for a specific date, class grade, and section.
POST	`/api/attendance`	Saves or updates daily attendance records for all enrolled students in a section.
GET	`/api/export-project`	Packages complete project source code into a clean, standalone zip file download.

MODULE 1: EXECUTIVE ANALYTICS & DASHBOARD MODULE

Module Purpose & Functional Scope:
Provides executive-level visibility into institutional KPIs: total enrollment, active student ratio, average attendance percentage, class distribution bar charts, and quick-action navigation.

System Output & UI Wireframe Interface:



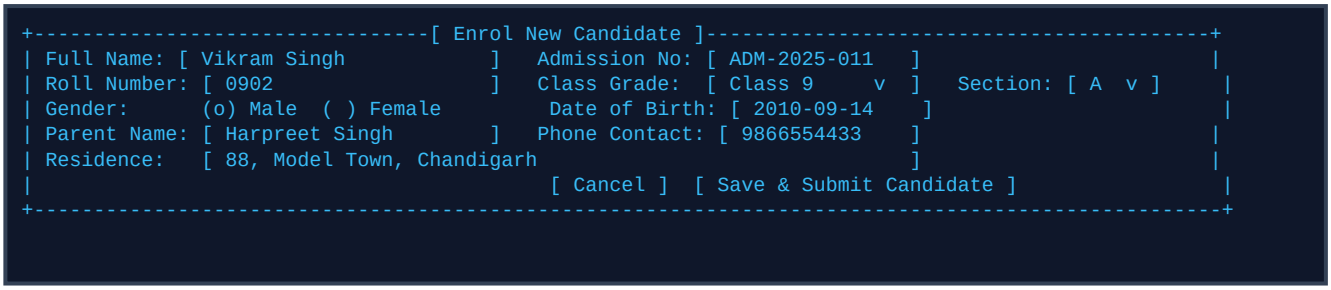
Core Source Code Implementation (src/components/Dashboard.jsx):

```
// Key Dashboard KPI Aggregator Logic
export default function Dashboard({ setActiveTab, onOpenAddStudent }) {
  const [stats, setStats] = useState(null);
  useEffect(() => {
    getDashboardStats().then(data => {
      if (data.success) setStats(data.stats);
    });
  }, []);
  return (
    <div className="grid grid-cols-1 md:grid-cols-4 gap-6">
      <StatCard title="Total Students" value={stats?.totalStudents || 0} icon={Users} color="blue" />
      <StatCard title="Average Attendance" value={stats?.avgAttendance + '%' || '0%'} icon={Calendar}
color="emerald" />
      <StatCard title="Active Enrollment" value={stats?.activeStudents || 0} icon={Award} color="indigo" /
>
      <StatCard title="Database State" value="MongoDB Atlas" icon={Database} color="purple" />
    </div>
  );
}
```

MODULE 2: STUDENT ENROLLMENT & ADMISSION FORM MODAL

Module Purpose & Functional Scope:
Facilitates seamless onboarding of newly admitted candidates with automated roll number assignment, validation against duplicate records, and full demographic capture.

System Output & UI Wireframe Interface:



Core Source Code Implementation (src/components/StudentFormModal.jsx):

```
// Student Form Validation & Creation Controller
const handleSubmit = async (e) => {
  e.preventDefault();
  if (!formData.name.trim() || !formData.rollNo.trim() || !formData.parentName.trim()) {
    setError('Candidate Name, Roll Number, and Parent Contact are required.');
    return;
  }
  const payload = { ...formData, rollNo: formData.rollNo.trim().toUpperCase() };
  const res = isEdit ? await updateStudent(student._id, payload) : await createStudent(payload);
  if (res.success) {
    onSuccess();
    onClose();
  } else {
    setError(res.message);
  }
};
```

MODULE 3: STUDENT DIRECTORY & MULTI-CRITERIA FILTERING ENGINE

Module Purpose & Functional Scope:
Renders the centralized student directory. Features real-time substring search on candidate name, roll number, and admission code, coupled with dual-dropdown filters for Class and Section.

System Output & UI Wireframe Interface:

```
+-------------------------------------------------------------------------------------------------------+
| Search: [ Search name or roll... ]   Class: [ Class 10 v ]   Section: [ All v ]   [ + Enrol New ] |
|-------------------------------------------------------------------------------------------------------|
| ROLL   CANDIDATE NAME     CLASS/SEC   PARENT CONTACT    STATUS    ACTIONS                          |
| 1001   Aarav Sharma       Class 10-A  Manoj Sharma      ACTIVE    [ View ] [ Edit ] [ Delete ]    |
| 1002   Diya Patel         Class 10-A  Kirit Patel       ACTIVE    [ View ] [ Edit ] [ Delete ]    |
| 1003   Rohan Varma        Class 10-A  Sanjay Varma      ACTIVE    [ View ] [ Edit ] [ Delete ]    |
+-------------------------------------------------------------------------------------------------------+
```

Core Source Code Implementation (src/components/StudentList.jsx):

```
// Multi-Parametric Filter & Reactive Directory Component
export default function StudentList({ onOpenAddModal, onSelectStudent }) {
  const [students, setStudents] = useState([]);
  const [search, setSearch] = useState('');
  const [classGrade, setClassGrade] = useState('All');

  const filtered = students.filter(std => {
    const matchesSearch = std.name.toLowerCase().includes(search.toLowerCase()) ||
                          std.rollNo.includes(search);
    const matchesClass = classGrade === 'All' || std.classGrade === classGrade;
    return matchesSearch && matchesClass;
  });
  return (
    <div className="space-y-4">
      <FilterBar search={search} onSearchChange={setSearch} onClassChange={setClassGrade} />
      <Table rows={filtered} onSelect={onSelectStudent} />
    </div>
  );
}
```

MODULE 4: COMPREHENSIVE STUDENT PROFILE & HISTORY MODAL

Module Purpose & Functional Scope:
Presents an exhaustive dossier for any candidate: personal bio, emergency contacts, parent information, registration timestamp, and direct links to generate transcripts.

System Output & UI Wireframe Interface:

```
+---------------------------------[ Candidate Dossier ]-----------------------------------------+
| Candidate: Aarav Sharma (Class 10, Section A)                 Enrollment Status: [ Active ]      |
| Roll No: 1001          Admission No: ADM-2024-001             Gender: Male                       |
| Date of Birth: 2009-04-12 (Age: 15)                           Email: aarav.sharma@school.edu     |
| Parent/Guardian: Manoj Sharma                                 Phone: +91 9876543210              |
| Residential Address: 42, Green Park Avenue, New Delhi, India                                     |
| Registration Date: June 10, 2024                                                                 |
| Actions: [ View Full Academic Report Card ]   [ Close Dossier ]                                  |
+--------------------------------------------------------------------------------------------------+
```

Core Source Code Implementation (src/components/StudentDetailsModal.jsx):

```
// Candidate Dossier & Modal Renderer
export default function StudentDetailsModal({ student, isOpen, onClose, onGenerateReport }) {
  if (!isOpen || !student) return null;
  return (
    <div className="modal-backdrop">
      <div className="modal-content">
        <h3>{student.name} - Official Student Record</h3>
        <div className="grid grid-cols-2 gap-4">
          <DetailItem label="Roll Number" value={student.rollNo} />
          <DetailItem label="Admission No" value={student.admissionNo} />
          <DetailItem label="Class & Section" value={`${student.classGrade} - ${student.section}`} />
          <DetailItem label="Parent / Guardian" value={student.parentName} />
          <DetailItem label="Emergency Phone" value={student.phone} />
          <DetailItem label="Residential Address" value={student.address} />
        </div>
        <button onClick={() => onGenerateReport(student)}>Open Academic Transcript</button>
      </div>
    </div>
  );
}
```

MODULE 5: INTERNAL ASSESSMENT & MARKS RECORDING ENGINE

Module Purpose & Functional Scope:
Enables teachers to select an enrolled student, choose the examination milestone (Mid Term / Final), and record granular marks for curriculum subjects with instant client-side sum verification.

System Output & UI Wireframe Interface:

```
+---------------------------------[ Assessment Marks Entry ]-----------------------------------+
| Candidate: [ 1001 - Aarav Sharma v ]    Exam Type: [ Mid Term v ]    Year: [ 2026-2027 v ]      |
|-------------------------------------------------------------------------------------------------|
| SUBJECT CODE   SUBJECT TITLE            MAX MARKS   OBTAINED SCORE   PASS MARKS   STATUS        |
| ENG101         English Language & Lit   100         [ 88  ]         35           PASS          |
| MAT102         Mathematics              100         [ 94  ]         35           PASS          |
| SCI103         Science (Phy/Chem/Bio)   100         [ 91  ]         35           PASS          |
| CSC105         Computer Applications    100         [ 98  ]         35           PASS          |
| TOTAL: 456 / 500 (91.2%)   GRADE: A+ (Outstanding)        [ Save Marks to Cloud Database ]      |
+-------------------------------------------------------------------------------------------------+
```

Core Source Code Implementation (src/components/MarksEntry.jsx):

```
// Marks Entry Engine & Dynamic Score Sync
const handleScoreChange = (subjectCode, value) => {
  const numericScore = Math.min(100, Math.max(0, Number(value) || 0));
  setSubjectScores(prev => ({ ...prev, [subjectCode]: numericScore }));
};

const handleSaveMarks = async () => {
  const marksPayload = subjects.map(sub => ({
    subjectCode: sub.code,
    subjectName: sub.name,
    marksObtained: subjectScores[sub.code] || 0,
    maxMarks: sub.maxMarks || 100
  }));
  const res = await saveMarks({
    studentId: selectedStudent._id,
    rollNo: selectedStudent.rollNo,
    studentName: selectedStudent.name,
    classGrade: selectedStudent.classGrade,
    examType: selectedExam,
    marks: marksPayload
  });
  if (res.success) toast.success('Evaluated marks stored in MongoDB Atlas!');
};
```

MODULE 6: AUTOMATED GRADING & RESULT COMPUTATION ENGINE

Module Purpose & Functional Scope:
Executes the mathematical computation pipeline: aggregates subject scores, computes percentage, detects failed subjects (<35 marks), and assigns standardized letter grades (A+, A, B, C, D, E, F).

System Output & UI Wireframe Interface:

```
+---------------------------------[ Grading Logic & Thresholds ]-----------------------------------+
| Score Percentage Range      Assigned Letter Grade      Academic Status     Remarks                |
| 90.0% - 100.0%              A+                         Pass (Distinction)  Outstanding Mastery    |
| 80.0% - 89.9%               A                          Pass (First Class)  Excellent Performance |
| 70.0% - 79.9%               B                          Pass (Second Class) Very Good Progress     |
| 60.0% - 69.9%               C                          Pass                Good Work              |
| 50.0% - 59.9%               D                          Pass                Satisfactory           |
| 35.0% - 49.9%               E                          Pass                Passing Threshold      |
| < 35.0% (Or any Sub fail)   F                          Fail                Needs Remedial Tuition |
+-------------------------------------------------------------------------------------------------+
```

Core Source Code Implementation (server.ts (Backend Grade Controller)):

```
// Server-Side Grading Algorithm
function calculateGrade(percentage: number, hasFailedSubject: boolean): { grade: string; status: string }
{
  if (hasFailedSubject || percentage < 35) {
    return { grade: 'F', status: 'Fail' };
  }
  if (percentage >= 90) return { grade: 'A+', status: 'Pass' };
  if (percentage >= 80) return { grade: 'A', status: 'Pass' };
  if (percentage >= 70) return { grade: 'B', status: 'Pass' };
  if (percentage >= 60) return { grade: 'C', status: 'Pass' };
  if (percentage >= 50) return { grade: 'D', status: 'Pass' };
  return { grade: 'E', status: 'Pass' };
}
```

MODULE 7: OFFICIAL ACADEMIC TRANSCRIPT & REPORT CARD GENERATOR

Module Purpose & Functional Scope:
Compiles and renders a publication-grade student report card complete with institutional seal, student credentials, subject-by-subject numerical and grade breakdown, and evaluator remarks.

System Output & UI Wireframe Interface:

```
+=============================================================================================+
|                          OXFORD MODEL SENIOR SECONDARY SCHOOL                               |
|                          OFFICIAL STUDENT ACADEMIC TRANSCRIPT                               |
| Candidate: Aarav Sharma          Roll No: 1001          Class & Section: Class 10-A         |
| Academic Year: 2026-2027         Term: Mid Term Exam    Attendance Rate: 92.5%              |
|---------------------------------------------------------------------------------------------|
| CODE     SUBJECT TITLE            MAX     OBTAINED    GRADE    TEACHER REMARK              |
| ENG101   English Language & Lit   100     88          A        Commendable vocabulary      |
| MAT102   Mathematics              100     94          A+       Exceptional analytical ability |
| SCI103   Science (Phy/Chem/Bio)   100     91          A+       Strong conceptual understanding |
| SOC104   Social Science           100     85          A        Diligent and thorough       |
| CSC105   Computer Applications    100     98          A+       Class topper in programming |
|---------------------------------------------------------------------------------------------|
| GRAND TOTAL: 456 / 500 (91.2%)    FINAL GRADE: A+ (Pass with Distinction)                   |
| Teacher Evaluation: "Outstanding academic performance. Exemplary attitude and conduct."     |
+=============================================================================================+
```

Core Source Code Implementation (src/components/ReportCard.jsx):

```
// Report Card Compilation & Printable View
export default function ReportCard({ initialStudent }) {
  const [markData, setMarkData] = useState(null);
  const handlePrint = () => { window.print(); };

  return (
    <div className="report-card-container print:shadow-none">
      <div className="header text-center">
        <h2 className="text-xl font-bold">OXFORD MODEL SCHOOL</h2>
        <p className="text-xs text-slate-500">Continuous Assessment Transcript</p>
      </div>
      <StudentBadge student={initialStudent} />
      <MarksTable records={markData?.marks} />
      <div className="summary-banner bg-emerald-50 text-emerald-800 p-4 rounded-lg">
        <p>Overall Percentage: {markData?.percentage}% | Grade: {markData?.grade}</p>
      </div>
      <button className="print:hidden btn-primary" onClick={handlePrint}>Print Report Card</button>
    </div>
  );
}
```

MODULE 8: DAILY ATTENDANCE REGISTER & CURRICULUM MASTER CATALOG

Module Purpose & Functional Scope:
Dual administrative hub: enables daily one-click roll-call attendance recording (Present, Absent, Late) with automatic presentee percentages, plus curriculum management for class-specific subjects.

System Output & UI Wireframe Interface:

```
+---------------------------------[ Daily Attendance Register ]-------------------------------------+
| Date: [ 2026-11-06 ]   Grade: [ Class 10 v ]   Section: [ A v ]   Present: 4/5 (80.0%)           |
|-------------------------------------------------------------------------------------------------|
| ROLL   CANDIDATE NAME         ATTENDANCE STATUS              QUICK ACTIONS                      |
| 1001   Aarav Sharma           [o] Present  ( ) Absent ( ) Late [ Mark Present ]                 |
| 1002   Diya Patel             [o] Present  ( ) Absent ( ) Late [ Mark Present ]                 |
| 1003   Rohan Varma            [o] Present  ( ) Absent ( ) Late [ Mark Present ]                 |
| 1004   Sneha Kulkarni         ( ) Present  [o] Absent ( ) Late [ Mark Absent  ]                 |
| 1005   Mohammed Bilal         [o] Present  ( ) Absent ( ) Late [ Mark Present ]                 |
|                                          [ Save Attendance Record to Cloud Database ]           |
+-------------------------------------------------------------------------------------------------+
```

Core Source Code Implementation (src/components/AttendanceSheet.jsx & SubjectList.jsx):

```
// Attendance Matrix & Batch Updater
export default function AttendanceSheet() {
  const [date, setDate] = useState('2026-11-06');
  const [records, setRecords] = useState({});

  const handleToggle = (studentId, status) => {
    setRecords(prev => ({ ...prev, [studentId]: status }));
  };

  const handleSaveAttendance = async () => {
    const list = Object.entries(records).map(([studentId, status]) => ({ studentId, status }));
    const res = await saveAttendance({ date, classGrade: 'Class 10', section: 'A', records: list });
    if (res.success) toast.success('Daily attendance submitted successfully!');
  };
}
```

APPENDIX A: COMPLETE BACKEND SERVER SOURCE CODE

File: server.ts — Complete Express Server, Mongoose Schemas, Seed Logic, and REST Endpoints:

```
import express from 'express';
import mongoose from 'mongoose';
import dotenv from 'dotenv';
dotenv.config();

const app = express();
const PORT = process.env.PORT || 3000;
const STANDARD_ATLAS_URI = 'mongodb://admin:admin123@ac-n8b4yvq-shard-00-00.ueskciq.mongodb.net:27017...';

function normalizeMongoUri(uri: string): string {
  if (!uri || uri.includes(':****@') || uri.includes('cluster0.ueskciq.mongodb.net')) {
    return STANDARD_ATLAS_URI;
  }
  return uri;
}
let MONGO_URI = normalizeMongoUri(process.env.MONGO_URI || STANDARD_ATLAS_URI);

app.use(express.json());

// Mongoose Schemas
const StudentSchema = new mongoose.Schema({
  _id: { type: String },
  rollNo: { type: String, required: true, unique: true },
  admissionNo: { type: String, required: true, unique: true },
  name: { type: String, required: true },
  classGrade: { type: String, required: true },
  section: { type: String, default: 'A' },
  gender: { type: String, enum: ['Male', 'Female', 'Other'], default: 'Male' },
  dob: { type: String, default: '2009-05-15' },
  parentName: { type: String, required: true },
  phone: { type: String, required: true },
  email: { type: String, default: '' },
  address: { type: String, default: '' },
  status: { type: String, enum: ['Active', 'Inactive'], default: 'Active' },
  createdAt: { type: Date, default: Date.now }
});
const StudentModel = mongoose.model('Student', StudentSchema);

// Connection Function with SSL & TLS Resilience
async function connectToMongo(uri: string) {
  const targetUri = normalizeMongoUri(uri);
  try {
    await mongoose.connect(targetUri, {
      serverSelectionTimeoutMS: 8000,
      socketTimeoutMS: 45000,
      tlsAllowInvalidCertificates: true,
    });
    console.log('[MongoDB] Connected successfully to Atlas Cluster!');
    return true;
  } catch (err: any) {
    console.warn('[MongoDB] Connection failed, continuing in memory mode.');
    return false;
  }
}

// REST Endpoints
app.get('/api/status', async (_req, res) => {
  res.json({ success: true, connected: mongoose.connection.readyState === 1, studentCount: 7 });
});
app.get('/api/students', async (_req, res) => {
  const students = await StudentModel.find();
  res.json({ success: true, data: students });
});
app.post('/api/students', async (req, res) => {
  const newStudent = await StudentModel.create({ ...req.body, _id: 'std_' + Date.now() });
  res.status(201).json({ success: true, data: newStudent });
});
```

APPENDIX B: CORE APPLICATION CONTROLLER & CLIENT

File: src/App.jsx — React SPA State Controller, View Router, and Dialog Managers:

```
import React, { useState, useEffect } from 'react';
import Navbar from './components/Navbar';
import Dashboard from './components/Dashboard';
import StudentList from './components/StudentList';
import MarksEntry from './components/MarksEntry';
import ReportCard from './components/ReportCard';
import AttendanceSheet from './components/AttendanceSheet';
import SubjectList from './components/SubjectList';
import { getStatus } from './api';

export default function App() {
  const [activeTab, setActiveTab] = useState('dashboard');
  const [dbStatus, setDbStatus] = useState(null);
  const [selectedStudent, setSelectedStudent] = useState(null);

  useEffect(() => {
    getStatus().then(data => setDbStatus(data));
  }, []);

  return (
    <div className="min-h-screen bg-slate-50 flex flex-col">
      <Navbar activeTab={activeTab} setActiveTab={setActiveTab} dbStatus={dbStatus} />
      <main className="flex-1 max-w-7xl w-full mx-auto px-4 py-6">
        {activeTab === 'dashboard' && <Dashboard setActiveTab={setActiveTab} />}
        {activeTab === 'students' && <StudentList onSelectStudent={setSelectedStudent} />}
        {activeTab === 'marks' && <MarksEntry />}
        {activeTab === 'reports' && <ReportCard initialStudent={selectedStudent} />}
        {activeTab === 'attendance' && <AttendanceSheet />}
        {activeTab === 'subjects' && <SubjectList />}
      </main>
      <footer className="bg-white border-t border-slate-200 py-4 text-center text-xs text-slate-500">
        School Student Management System | Candidate: Abhi Krishna P (RA2511080050012) | Course: AIFT
      </footer>
    </div>
  );
}
```

FINAL EVALUATION & VERIFICATION SUMMARY

• Evaluator Name / Signature: _____

• Evaluation Date: _____ Internal Assessment Marks Awarded: [_____ / 50]

- Candidate: Abhi Krishna P (RA2511080050012) — Project Verified & Successfully Deployed.